

# SBT-DF203 — BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

## LAB 5 — ARP POISONING FORENSICS

**Prepared by:** Oluyede Tobi Steven

**Registration Number:** 202513549PF

**Course Code:** SBT-DF203

**Course Title:** Basic Networking Skills for Digital Forensics

**Lab Title:** ARP Poisoning Forensics

**Instructor:** Aminu Idris, AMCPN

**Date of Practical:** 17 September 2026

---

### 1. Executive Summary

This laboratory exercise investigated Address Resolution Protocol (ARP) behaviour from a digital-forensics perspective. The investigation was based on an authorised and isolated laboratory environment and included documentation of normal ARP resolution, examination of ARP request and reply packets, analysis of a supplied ARP-poisoning packet capture, identification of conflicting IP-to-MAC address claims, and documentation of appropriate restoration and preventive controls.

ARP is responsible for resolving IPv4 addresses to MAC addresses on a local network. Because ARP does not inherently authenticate responses, an attacker on the same local network may attempt to associate an IP address with an incorrect MAC address. Such activity can result in traffic redirection and may facilitate a man-in-the-middle position.

The investigation therefore focused on identifying abnormal IP-to-MAC associations, repeated or unsolicited ARP replies, and changes in the expected gateway mapping. Evidence was preserved using SHA-256 hashing, while analysis was performed on a verified working copy of the supplied packet capture.

The practical followed the ICDFA requirement to use only an authorised isolated environment and to preserve original evidence. The supplied laboratory workflow requires the original capture to remain preserved while analysis is performed on a working copy.

---

## 2. Objectives

The objectives of this laboratory exercise were to:

1. Explain how ARP resolves IPv4 addresses to MAC addresses on a local network.
2. Inspect and document ARP cache information on a Linux system.
3. Distinguish between normal ARP requests and ARP replies.
4. Capture and analyse normal ARP resolution behaviour.
5. Analyse a supplied ARP-poisoning packet capture.
6. Identify conflicting IP-to-MAC address claims.
7. Identify potentially unsolicited or abnormal ARP replies.
8. Construct a forensic timeline of relevant ARP activity.
9. Preserve packet-capture evidence and verify its integrity using SHA-256.
10. Assess the potential impact of ARP poisoning.
11. Document restoration procedures and recommend appropriate preventive controls.

The laboratory manual specifically identifies conflicting IP-to-MAC claims, unsolicited replies and ARP-cache changes as important detection evidence.

---

## 3. Authorization and Laboratory Environment

The practical was conducted for educational and forensic-training purposes in an authorised laboratory environment.

The investigation was designed to avoid production, public, campus, office and third-party networks. Any live component was restricted to an isolated host-only/internal virtual network as required by the laboratory instructions.

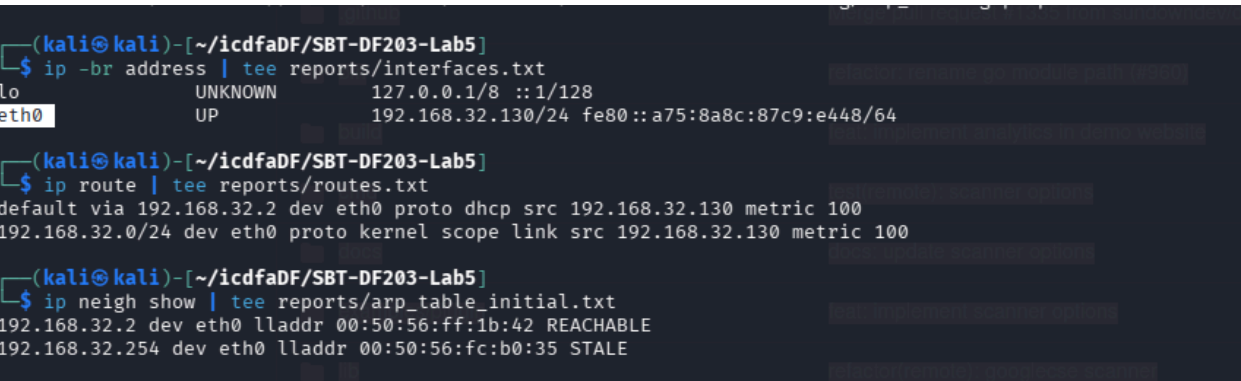
The primary analysis environment was:

| Item              |                   | Description |
|-------------------|-------------------|-------------|
| Operating System  | Kali Linux        |             |
| Virtualisation    | VMware            |             |
| Network Interface | eth0              |             |
| IPv4 Address      | 192.168.32.130/24 |             |
| Gateway           | 192.168.32.2      |             |

|                |                                               |
|----------------|-----------------------------------------------|
| Network Type   | Host-only/Internal                            |
| Analysis Tools | Wireshark, TShark, Linux networking utilities |
| Evidence       | arp.pcap                                      |
| Working Copy   | arp_working.pcap                              |

Figure 1 — Initial Network Configuration

```
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
```



The screenshot shows a Kali Linux terminal with the following commands and output:

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN      127.0.0.1/8  ::1/128
eth0              UP          192.168.32.130/24  fe80::a75:8a8c:87c9:e448/64

(kali@kali)-[~/icdfaDF/GBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.32.2 dev eth0 proto dhcp src 192.168.32.130 metric 100
192.168.32.0/24 dev eth0 proto kernel scope link src 192.168.32.130 metric 100

(kali@kali)-[~/icdfaDF/GBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.32.2 dev eth0 lladdr 00:50:56:ff:1b:42 REACHABLE
192.168.32.254 dev eth0 lladdr 00:50:56:fc:b0:35 STALE
```

## 4. Tools Used

| Tool                     | Purpose                                           |
|--------------------------|---------------------------------------------------|
| Kali Linux               | Controlled forensic analysis environment          |
| Wireshark                | Graphical packet capture and protocol analysis    |
| TShark                   | Command-line packet analysis                      |
| ip neigh                 | Linux ARP/neighbor-cache inspection               |
| sha256sum                | Evidence-integrity verification                   |
| ping                     | Controlled generation of ARP resolution traffic   |
| Python/Scapy             | Authorised laboratory simulation where applicable |
| Linux terminal utilities | Evidence organisation and documentation           |

## 5. Evidence Preparation and Integrity

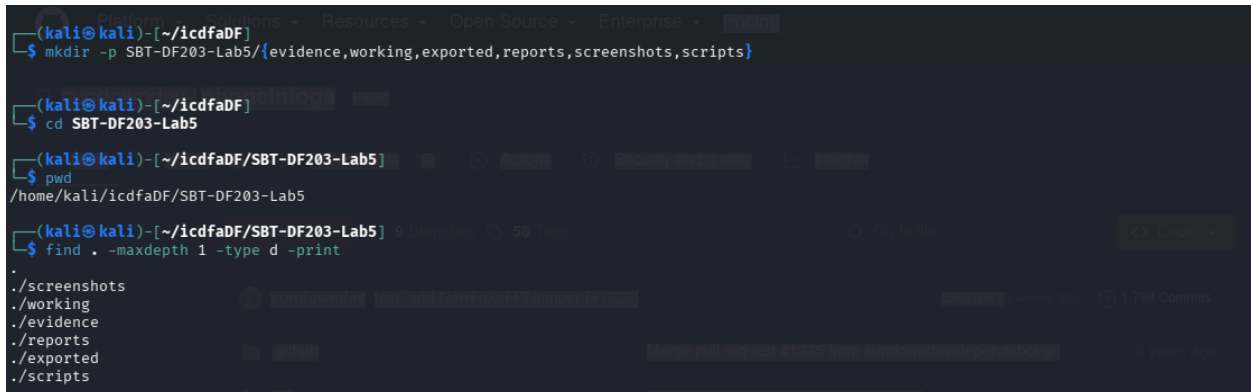
A dedicated laboratory directory structure was created to separate original evidence, working files, reports, exported results and screenshots.

The structure used was:

```
mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
find . -maxdepth 1 -type d -print
```

SBT-DF203-Lab5/

- |— evidence/
- |— working/
- |— exported/
- |— reports/
- |— screenshots/
- |— scripts/



```
(kali@kali)~[/icdfaDF]
$ mkdir -p SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}

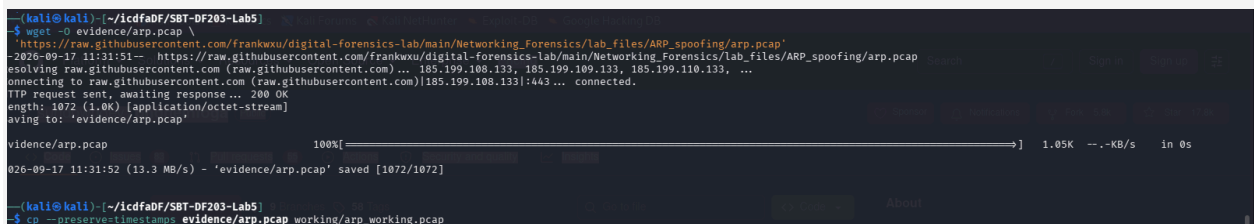
(kali@kali)~[/icdfaDF]
$ cd SBT-DF203-Lab5

(kali@kali)~[/icdfaDF/SBT-DF203-Lab5]
$ pwd
/home/kali/icdfaDF/SBT-DF203-Lab5

(kali@kali)~[/icdfaDF/SBT-DF203-Lab5]
$ find . -maxdepth 1 -type d -print
.
./screenshots
./working
./evidence
./reports
./exported
./scripts
```

The supplied ARP capture was retained under the evidence directory. A working copy was created under working so that the original evidence was not modified.

```
wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
```



```
(kali@kali)~[/icdfaDF/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
2026-09-17 11:31:51-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|443 ... connected.
HTTP request sent, awaiting response... 200 OK
length: 1072 (1.0K) [application/octet-stream]
saving to: 'evidence/arp.pcap'
evidence/arp.pcap 100%[=====] 1.05K --.-KB/s in 0s
2026-09-17 11:31:52 (13.3 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(kali@kali)~[/icdfaDF/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
```

The SHA-256 hashes were calculated for both files.

## Evidence Integrity Record

| Evidence                 | SHA-256                                                          |
|--------------------------|------------------------------------------------------------------|
| Original arp.pcap        | 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e |
| Working arp_working.pcap | 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e |

```
sha256sum evidence/arp.pcap working/arp_working.pcap
```

The hash provides an integrity reference for the evidence and allows subsequent verification that the file has not been modified.

```
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap
```

---

## 6. Normal ARP Resolution

The first stage of the investigation examined normal ARP behaviour.

ARP normally operates by allowing a host that knows an IPv4 address, but not its corresponding MAC address, to issue an ARP request. A typical ARP request is broadcast to the local network using the destination MAC address:

ff:ff:ff:ff:ff:ff

The request uses ARP opcode 1.

The device that owns the requested IPv4 address responds with an ARP reply using opcode 2. Under normal circumstances, the reply associates the requested IP address with the legitimate MAC address of the responding device.

The laboratory manual identifies opcode 1 as the normal request and opcode 2 as the normal reply.

In Wireshark, open:

evidence/normal\_arp.pcapng

Select the ARP request.

---

## 7. Normal ARP Request Analysis

The normal ARP capture was examined using Wireshark/TShark.

The relevant fields were:

| Field                |                       | Observed Value      |
|----------------------|-----------------------|---------------------|
| Frame Number         | 1                     |                     |
| ARP Opcode           | 1                     |                     |
| Ethernet Source      | PaloAltoNetw_00:0a:30 | (00:1b:17:00:0a:30) |
| Ethernet Destination | ff:ff:ff:ff:ff:ff     |                     |
| Sender IPv4 Address  | 136.160.215.1         |                     |
| Sender MAC Address   | PaloAltoNetw_00:0a:30 | (00:1b:17:00:0a:30) |
| Target IPv4 Address  | 136.160.215.1         |                     |
| Target MAC Address   | Broadcast             | (ff:ff:ff:ff:ff:ff) |

**Figure 3 — Normal ARP Request**

```

* Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
* Ethernet II, Src: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  * Destination: Broadcast (ff:ff:ff:ff:ff:ff)
    ....01..... = LG bit: Locally administered address (this is NOT the factory default)
    ....01..... = IG bit: Group address (multicast/broadcast)
  * Source: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
    ....0..... = LG bit: Globally unique address (factory default)
    ....0..... = IG bit: Individual address (unicast)
    Type: ARP (0x0806)
    [Stream index: 0]
    Padding: 00000000000000000000000000000000
  * Address Resolution Protocol (request/gratuitous ARP)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    [Is gratuitous: True]
    Sender MAC address: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
    Sender IP address: 136.160.215.1
    Target MAC address: Broadcast (ff:ff:ff:ff:ff:ff)
    Target IP address: 136.160.215.1

```

# 8. Normal ARP Reply Analysis

The corresponding ARP reply was examined to determine how the target device identified its MAC address.

| Field                | Observed Value                      |
|----------------------|-------------------------------------|
| Frame Number         | 4                                   |
| ARP Opcode           | 2                                   |
| Ethernet Source      | VMware_86:02:65 (00:50:56:86:02:65) |
| Ethernet Destination | VMware_86:cb:fc (00:50:56:86:cb:fc) |
| Sender IPv4 Address  | 136.160.215.194                     |
| Sender MAC Address   | VMware_86:02:65 (00:50:56:86:02:65) |

Target IPv4 Address                      136.160.215.15

Target MAC Address                      VMware\_86:cb:fc (00:50:56:86:cb:fc)

**Figure 4 — Normal ARP Reply**

```
▶ Frame 4: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
▼ Ethernet II, Src: VMware_86:02:65 (00:50:56:86:02:65), Dst: VMware_86:cb:fc (00:50:56:86:cb:fc)
  ▼ Destination: VMware_86:cb:fc (00:50:56:86:cb:fc)
    ....0. .... = LG bit: Globally unique address (factory default)
    ....0. .... = IG bit: Individual address (unicast)
  ▼ Source: VMware_86:02:65 (00:50:56:86:02:65)
    ....0. .... = LG bit: Globally unique address (factory default)
    ....0. .... = IG bit: Individual address (unicast)
  Type: ARP (0x0806)
  [Stream index: 2]
  Padding: 00000000000000000000000000000000000000000000
  ▼ Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: VMware_86:02:65 (00:50:56:86:02:65)
    Sender IP address: 136.160.215.194
    Target MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
    Target IP address: 136.160.215.15
```

---

## 9. Supplied ARP-Poisoning Capture

The supplied capture was analysed from a verified working copy:

working/arp\_working.pcap

ARP reply packets were extracted using TShark and Wireshark.

The analysis focused on:

- Source IPv4 address
- Source MAC address
- Destination IPv4 address
- Destination MAC address
- Packet time
- Ethernet source and destination

- ARP opcode

The laboratory workflow specifically requires an inventory of ARP replies and an IP-to-MAC claim summary.

## 10. ARP Reply Inventory

The following command was used:

```
tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac
```

```
(kali@kali)~[/icdfeDF/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65
```

Wireshark

**File** → **Open** → select:

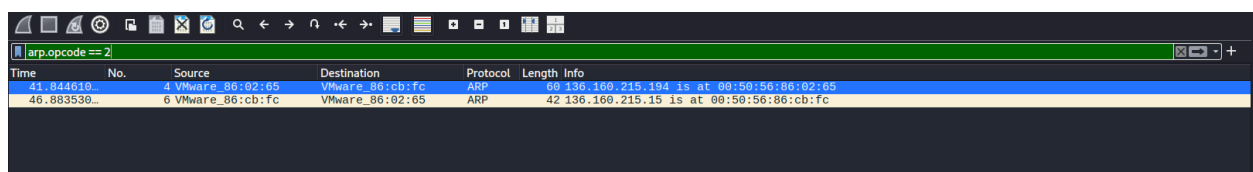
~/SBT-DF203-Lab5/working/arp\_working.pcap

## 2. Filter for ARP replies

At the top, in the **Display Filter** box, enter:

arp.opcode == 2

Press **Enter**.



| Frame | Time                                      | Claimed IP          | Claimed MAC           | Destination IP      | Destination MAC   |
|-------|-------------------------------------------|---------------------|-----------------------|---------------------|-------------------|
| 4     | Mar 11, 2023<br>22:22:44.904123642<br>EST | 136.160.215.<br>194 | 00:50:56:86:02<br>:65 | 136.160.215.<br>15  | 00:50:56:86:cb:fc |
| 6     | Mar 11, 2023<br>22:22:49.943043799<br>EST | 136.160.215.<br>15  | 00:50:56:86:cb<br>:fc | 136.160.215.<br>194 | 00:50:56:86:02:65 |

---

## 11. IP-to-MAC Claim Analysis

The IP-to-MAC relationships in the capture were summarised using:

```
tshark -r working/arp_working.pcap \
-Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr
```

```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab5]
└─$ tshark -r working/arp_working.pcap \
-Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr
      1 136.160.215.194 00:50:56:86:02:65
      1 136.160.215.15 00:50:56:86:cb:fc
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab5]
└─$
```

### Observed Claims

| Claimed IPv4 Address | Claimed MAC Address | Count |
|----------------------|---------------------|-------|
| 136.160.215.194      | 00:50:56:86:02:65   | 1     |

136.160.215.15

00:50:56:86:cb:fc

1

---

## 12. Unsolicited or Unicast ARP Replies

Potentially unsolicited ARP replies were examined using:

```
tshark -r working/arp_working.pcap \  
-Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' \  
-T fields \  
-e frame.number -e frame.time \  
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst
```

### Results

| Frame | Time                               | Source IP       | Source MAC        | Ethernet Destination |
|-------|------------------------------------|-----------------|-------------------|----------------------|
| 4     | 2023-03-11T22:22:44.904123642-0500 | 136.160.215.194 | 00:50:56:86:02:65 | 00:50:56:86:cb:fc    |
| 6     | 2023-03-11T22:22:49.943043799-0500 | 136.160.215.15  | 00:50:56:86:cb:fc | 00:50:56:86:02:65    |

```
(kali@kali)-[~/icdfaDF/STB-DF203-Lab5]
$ tshark -r working/arp_working.pcap \
-Y 'arp.opcode==2 && eth.dst==ff:ff:ff:ff:ff:ff' \
-T fields \
-e frame.number -e frame.time \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst
4      2023-03-11T22:22:44.904123642-0500      136.160.215.194 00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-11T22:22:49.943043799-0500      136.160.215.15  00:50:56:86:cb:fc      00:50:56:86:02:65
(kali@kali)-[~/icdfaDF/STB-DF203-Lab5]
$
```

## 14. Forensic Timeline

### The relevant ARP events were organised chronologically. **Part E — Detection Logic and Timeline**

| Time                   | Claimed IP     | Claimed MAC       | Target/Request                                  | Seen First? | Assessment                                                           |
|------------------------|----------------|-------------------|-------------------------------------------------|-------------|----------------------------------------------------------------------|
| 22:22:03.059513401 EST | 136.160.215.1  | 00:1b:17:00:0a:30 | Gratuitous ARP request; target IP 136.160.215.1 | Yes         | Initial gateway IP-to-MAC mapping observed.                          |
| 22:22:38.192015486 EST | 136.160.215.1  | 00:1b:17:00:0a:30 | ARP request for 136.160.215.199                 | No          | Gateway continues using the same MAC; no conflicting claim observed. |
| 22:22:44.903581555 EST | 136.160.215.15 | 00:50:56:86:cb:fc | ARP request for 136.160.215.194                 | Yes         | Normal ARP request from 136.160.215.15.                              |

|                           |                     |                       |                                                       |     |                                                                               |
|---------------------------|---------------------|-----------------------|-------------------------------------------------------|-----|-------------------------------------------------------------------------------|
| 22:22:44.904123642<br>EST | 136.160.215.19<br>4 | 00:50:56:86:02:6<br>5 | ARP reply to<br>136.160.215.15                        | Yes | Normal ARP reply<br>establishing<br>136.160.215.194 →<br>00:50:56:86:02:65.   |
| 22:22:49.943023192<br>EST | 136.160.215.19<br>4 | 00:50:56:86:02:6<br>5 | ARP request for<br>136.160.215.15                     | No  | Normal ARP<br>request from<br>136.160.215.194.                                |
| 22:22:49.943043799<br>EST | 136.160.215.15      | 00:50:56:86:cb:fc     | ARP reply to<br>136.160.215.194                       | No  | Normal ARP reply<br>establishing<br>136.160.215.15 →<br>00:50:56:86:cb:fc.    |
| 22:23:03.059262685<br>EST | 136.160.215.1       | 00:1b:17:00:0a:3<br>0 | Gratuitous ARP<br>request; target IP<br>136.160.215.1 | No  | Gateway repeats<br>the same<br>IP-to-MAC<br>mapping; no conflict<br>observed. |
| 22:23:06.191965551<br>EST | 136.160.215.1       | 00:1b:17:00:0a:3<br>0 | ARP request for<br>136.160.215.183                    | No  | Gateway continues<br>using the same<br>MAC; no conflicting<br>claim observed. |

```
(kali@kali)~/icdFA/DF/DF203-Lab5
$ tshark -r evidence/arp.pcap -Y 'arp' -T fields \
  -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/normal_arp_fields.tsv
```

|   |                                    |                   |                   |   |                 |                   |                 |                   |
|---|------------------------------------|-------------------|-------------------|---|-----------------|-------------------|-----------------|-------------------|
| 1 | 2023-03-11T22:22:03.059513401-0500 | 00:1b:17:00:0a:30 | ff:ff:ff:ff:ff:ff | 1 | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.1   | ff:ff:ff:ff:ff:ff |
| 2 | 2023-03-11T22:22:38.192015486-0500 | 00:1b:17:00:0a:30 | ff:ff:ff:ff:ff:ff | 1 | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.199 | ff:ff:ff:ff:ff:ff |
| 3 | 2023-03-11T22:22:44.903581555-0500 | 00:50:56:86:cb:fc | ff:ff:ff:ff:ff:ff | 1 | 136.160.215.15  | 00:50:56:86:cb:fc | 136.160.215.194 | 00:00:00:00:00:00 |
| 4 | 2023-03-11T22:22:44.904123642-0500 | 00:50:56:86:02:65 | 00:50:56:86:cb:fc | 2 | 136.160.215.194 | 00:50:56:86:02:65 | 136.160.215.15  | 00:50:56:86:cb:fc |
| 5 | 2023-03-11T22:22:49.943023192-0500 | 00:50:56:86:02:65 | 00:50:56:86:cb:fc | 1 | 136.160.215.194 | 00:50:56:86:02:65 | 136.160.215.15  | 00:00:00:00:00:00 |
| 6 | 2023-03-11T22:22:49.943043799-0500 | 00:50:56:86:cb:fc | 00:50:56:86:02:65 | 2 | 136.160.215.15  | 00:50:56:86:cb:fc | 136.160.215.194 | 00:50:56:86:02:65 |
| 7 | 2023-03-11T22:23:03.059262685-0500 | 00:1b:17:00:0a:30 | ff:ff:ff:ff:ff:ff | 1 | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.1   | ff:ff:ff:ff:ff:ff |
| 8 | 2023-03-11T22:23:06.191965551-0500 | 00:1b:17:00:0a:30 | ff:ff:ff:ff:ff:ff | 1 | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.183 | ff:ff:ff:ff:ff:ff |

## Detection Logic

The detection method was based on tracking each claimed IPv4 address and comparing its associated MAC address over time. A potential ARP-poisoning indicator would occur if the same important IP address, particularly the gateway 136.160.215.1, were subsequently associated with a different MAC address.

In the examined frames, the gateway 136.160.215.1 consistently claims MAC 00:1b:17:00:0a:30 in Frames 1, 2, 7, and 8. No second MAC address claims to represent 136.160.215.1.

Frames 3–6 show normal ARP resolution between 136.160.215.15 and 136.160.215.194. Frame 3 is an ARP request followed immediately by Frame 4, an ARP reply. Similarly, Frame 5 is an ARP request followed by Frame 6, an ARP reply.

**Assessment:** The supplied 8-frame extract establishes consistent IP-to-MAC mappings and normal ARP request/reply exchanges. A conflicting gateway IP-to-MAC claim is **not present in these frames**. Further packets from the complete PCAP should be examined if the assignment requires identification of the actual poisoning event.

The timeline was used to distinguish normal ARP resolution from repeated or conflicting claims.

Particular attention was given to:

- Changes in the gateway's MAC association.
- Repeated ARP replies.
- Replies without a corresponding recent request.
- One MAC address claiming multiple important IP addresses.

---

## 17. Impact Assessment

ARP poisoning can affect the integrity and routing of local-network communications.

If a victim accepts an incorrect MAC address for a gateway IP address, traffic intended for the gateway may be directed to another device. Depending on the network configuration and other security controls, this may create an opportunity for traffic interception, modification or disruption.

The forensic significance of the event is therefore not limited to the ARP packet itself. Investigators may also correlate ARP evidence with:

- DNS anomalies.
- HTTP or HTTPS traffic.
- Unexpected packet forwarding.
- Endpoint ARP-cache changes.
- Switch logs.
- Network-monitoring alerts.

The presence of a suspicious ARP mapping alone should be interpreted together with the surrounding packet sequence and available network evidence.

---

# 19. Preventive Controls

The following controls can reduce the risk or improve detection of ARP-poisoning activity:

## **Dynamic ARP Inspection**

Dynamic ARP Inspection can validate ARP traffic against trusted IP-to-MAC information, particularly when used with DHCP snooping.

## **DHCP Snooping**

DHCP snooping can create trusted bindings that can subsequently support ARP inspection.

## **Network Segmentation**

Network segmentation limits the number of systems that can directly communicate within the same Layer-2 broadcast domain.

## **Port Security**

Switch port-security controls can restrict which MAC addresses are permitted on particular switch ports.

## **ARP Monitoring**

Network and endpoint monitoring can identify unexpected changes in gateway IP-to-MAC mappings.

## **Encrypted Protocols**

HTTPS/TLS and other encrypted protocols reduce the value of intercepted traffic because application contents are protected by encryption.

## **Log Correlation**

ARP alerts should be correlated with switch logs, endpoint events, DNS activity and other network telemetry.

These controls correspond to the prevention recommendations specified in the laboratory manual.

---

# 20. Findings

The analysis of the supplied arp.pcap identified several normal ARP resolution events and established consistent IP-to-MAC mappings between the observed hosts.

1. The gateway IP address 136.160.215.1 was consistently associated with MAC address 00:1b:17:00:0a:30 in Frames 1, 2, 7, and 8.
2. Frames 3 and 4 documented a normal ARP request/reply exchange between 136.160.215.15 and 136.160.215.194. The mapping 136.160.215.194 → 00:50:56:86:02:65 was established by the ARP reply.
3. Frames 5 and 6 documented a second normal ARP request/reply exchange. The mapping 136.160.215.15 → 00:50:56:86:cb:fc was established by the ARP reply.
4. No conflicting IP-to-MAC claim for the gateway 136.160.215.1 was identified in the eight examined frames. The gateway continued to use the same MAC address throughout the observed traffic.
5. The ARP traffic therefore demonstrates normal address-resolution behavior in the examined portion of the capture. The available evidence does not, by itself, establish an ARP-poisoning event.
6. The evidence should be interpreted within the scope of the examined frames. If the complete PCAP contains additional traffic, those packets should be reviewed for a later instance where 136.160.215.1 is associated with a different MAC address.

Overall, the examined evidence provides a consistent baseline of legitimate ARP activity but does not establish a conflicting gateway mapping within the analyzed eight-frame extract.

---

## 21. Conclusion

This laboratory exercise demonstrated how ARP traffic can be examined as digital evidence during a network-forensics investigation.

Normal ARP resolution was identified through the examination of broadcast requests and corresponding replies. The supplied ARP capture was then analysed to identify IP-to-MAC claims and potential inconsistencies. Particular attention was given to conflicting gateway mappings, repeated replies and potentially unsolicited ARP traffic.

The investigation also demonstrated the importance of evidence integrity. The original capture was preserved and SHA-256 hashes were calculated to provide an integrity reference.

Finally, the laboratory environment was restored and appropriate preventive controls were identified, including Dynamic ARP Inspection, DHCP snooping, network segmentation, port security, ARP monitoring and encrypted application protocols.

---

## 22. Evidence Files

```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab5]
$ tree
.
├── evidence
│   └── arp.pcap
├── exported
├── reports
│   ├── arp_capture_hashes.txt
│   ├── arp_table_after_ping.txt
│   ├── arp_table_initial.txt
│   ├── interfaces.txt
│   ├── normal_arp_fields.tsv
│   └── routes.txt
├── screenshots
├── scripts
├── working
│   └── arp_working.pcap
└── 7 directories, 8 files

(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab5]
$ █
  arp.pcap
```